

Computação Científica e Análise de Dados
Regressão Logística Para Estimativa De Faixa Etária

Yuri Rocha de Albuquerque

123166143

Rio, 27 de novembro de 2024

Motivação e Objetivo

Desde o começo das aplicações vistas na disciplina, fiquei impressionado com as infinitas possibilidades de uso dos algoritmos, especialmente na área de inteligência artificial. Dito isso, quando comecei a ver as aplicações dos conteúdos nessa área, fiquei muito empolgado para fazer algum projeto que envolvesse aprendizado supervisionado ou não supervisionado apenas utilizando o que foi ensinado em sala. A partir disso, surgiu a ideia de fazer um classificador de fotos de pessoas com base na faixa etária que aparentam ter utilizando regressão logística, e foi essa a ideia que tentei desenvolver e que descreverei ao decorrer desse relatório.

Planejamento inicial

De início, precisava de um *dataset* que contivesse, pelo menos, milhares de fotos de pessoas para que o modelo ficasse minimamente preciso. Então, utilizei o dataset denominado *UTKFace*, disponibilizado através de um site hospedado pelo github (Link nas referências). A partir daí, comecei a planejar que tipo de outros métodos usaria e como se daria a organização dos dados. Cheguei à conclusão de que seria necessário também a utilização do PCA para a redução da complexidade das matrizes no algoritmo e, conseqüentemente, a velocidade do mesmo, devido ao número muito grande de linhas e colunas que viriam a ter. Após a definição dos métodos e com o dataset pronto, comecei a montar o código do programa.

Solução

Primeiramente, depois de baixar o arquivo com as fotos, criei uma pasta para guardar as fotos das crianças (0-12 anos), outra para guardar as fotos dos adolescentes (13-17 anos), outra pra guardar fotos de adultos (18-64 anos) e a última para guardar as fotos dos idosos (65 anos ou mais), que serviriam mais tarde para atribuir *labels* a essas fotos. Vale ressaltar que tentei subdividir a faixa etária dos adultos em duas outras: jovem adulto e adulto maduro, porém, por limitações de hardware e quantidade de dados insuficientes, decidi juntar os dois em uma só categoria para não afetar muito a performance e nem a precisão da solução.

Após isso, comecei a de fato trabalhar na solução. A primeira função criada foi a de carregamento de dados, já que tinha que guardar as fotos em uma matriz e atribuir *labels* as mesmas para dar como entrada ao algoritmo. Antes de guardar

essas imagens em uma matriz, apliquei um filtro nas mesmas para realçar suas características: foi usado um filtro de suavização leve (para retirada de ruídos) e outro de aumento de contraste (para acentuar as bordas), e o resultado pode ser conferido a seguir:



Imagem sem o filtro



Imagem com o filtro

Percebe-se, ao observar as fotos, que a imagem com o filtro realçou as rugas da pessoa na foto (deixando-as mais evidentes em preto), e eliminou levemente os ruídos que poderiam interferir na classificação, o que serviu para melhorar a precisão do algoritmo. Depois disso, linearizei as imagens e as guardei em uma

matriz, seguido do armazenamento também da *label* correspondente a cada foto em uma lista de *labels*. Um detalhe importante de mencionar é que foram usadas 16627 fotos e cada uma foi redimensionada para 64x64 pixels no algoritmo, para o equilíbrio entre performance e precisão. Logo, a matriz com a fotos linearizadas possui 16627 linhas e 4096 colunas (64^2 colunas).

Após guardar as informações na matriz, separei 25% dos dados para teste e o restante para treino do modelo de regressão logística. Depois disso, apliquei o PCA para a redução de dimensionalidade de forma que o algoritmo mantivesse 98% da variância nos dados (no Python é muito fácil conseguir isso visto que, ao invés de passar a quantidade de componentes que você deseja utilizar, há a opção de passar uma porcentagem como parâmetro que, nesse caso, significa a porcentagem da variância que você deseja manter, eliminando a necessidade de análise de gráficos e aplicação de métodos como o do cotovelo), e o resultado foi que houve uma redução de 4096 para 316 colunas, apenas. Uma imagem reconstruída pode ser observada a seguir:



Imagem original



Imagem reconstruída com o PCA

Percebe-se que as principais características da pessoa na foto foram mantidas (tamanho dos olhos, formato dos lábios, formato do rosto, formato do maxilar, etc), mesmo que em detrimento de alguns detalhes que, para essa análise, são irrelevantes (como cor da pele).

Após o pré-processamento das imagens, alimentei o modelo de Regressão Logística Multinomial com os dados separados para teste, especificando uma quantidade de iterações igual a 1000 (que já foi o suficiente para a convergência, por conta da normalização e baixa dimensão). No código, `X_train_pca` é a parte dos dados separados para teste reduzidos em dimensões pelo PCA, e `y_train` é a lista de labels separadas para teste:

```
modelo = LogisticRegression(solver='lbfgs', max_iter=1000)
modelo.fit(X_train_pca, y_train)
```

▼ **LogisticRegression** ⓘ ?

LogisticRegression(max_iter=1000)

Observação: a utilização do modelo Multinomial em detrimento do Binomial se deu pois, para a minha classificação, há mais do que dois rótulos possíveis para as imagens, o que me impossibilitou de usar o Binomial.

Regressão Logística Multinomial

Antes de continuar, vale entender resumidamente como o modelo de regressão logística multinomial funciona. Essa regressão é uma extensão da binária, funcionando em problemas de classificação multiclasse. Essencialmente, faz a mesma coisa que o modelo binomial, mas com algumas diferenças.

Para fins de exemplo, seja a função sigmoide:

$$\frac{1}{1 + e^{-z}}$$

Onde $z = c_1 + c_2 x$.

Para cada categoria K (onde $K = 1, 2, 3, \dots$), é criado K modelos binários, onde um deles serve como “base” para os outros, de forma que os coeficientes c_1 e c_2 calculados usando o método do gradiente descendente para o modelo base sejam subtraídos dos outros coeficientes calculados para os outros modelos, de forma que cada um modelo tenha novos coeficientes c'_i e c'_j . Por exemplo:

Se o modelo 1 $\frac{1}{1+e^{-(c_1+c_2x)}} = \frac{1}{1+e^{-z_1}}$ for usado como base para um outro modelo 2 $\frac{1}{1+e^{-(c_3+c_4x)}} = \frac{1}{1+e^{-z_2}}$, os novos coeficientes (para ambos os modelos) virariam $\frac{1}{1+e^{-((c_1-c_1)+(c_2-c_2)x)}} = \frac{1}{1+e^{-z'_1}}$ para o modelo 1 e $\frac{1}{1+e^{-((c_3-c_1)+(c_4-c_2)x)}} = \frac{1}{1+e^{-z'_2}}$ para o modelo 2.

Após a transformação nos coeficientes, é aplicada a função softmax para calcular a que grupo um novo dado pertence, sendo a função softmax definida por:

$$s(z'_i) = \frac{e^{z'_i}}{\sum e^{z'_j}}$$

Perceba que, se z'_1 corresponder ao z modificado do modelo base de um total de n modelos, a função ficará com o formato:

$$s(z'_i) = \frac{e^{z'_i}}{1 + e^{z'_1} + e^{z'_2} + \dots + e^{z'_n}}$$

Após isso, o valor das entradas do novo dado é substituído no x dos z' e é calculado uma proporção, que corresponde à probabilidade de um novo dado com entradas x estar no grupo i , sendo esse grupo definido como o maior valor entre todos os $s(z'_i)$ calculados.

Resultados

Com o modelo treinado, o apliquei nos dados separados para validação e obtive os seguintes resultados:

Relatório de Classificação:				
	precision	recall	f1-score	support
0-12_infantil	0.83	0.82	0.83	853
13-17_adolescente	0.28	0.08	0.12	205
18-64_adulto	0.83	0.92	0.87	2612
65+_idoso	0.68	0.51	0.58	487
accuracy			0.81	4157
macro avg	0.65	0.58	0.60	4157
weighted avg	0.78	0.81	0.79	4157

Percebe-se que o modelo classificou muito bem as imagens de crianças e de adultos, de maneira mediana as imagens de idosos e de maneira ruim as de adolescentes. Isso se deve ao fato de que, para crianças, adultos e idosos, possuo uma quantidade muito grande de fotos, o que ajudou o modelo a classificá-los melhor. Para os adolescentes, além de não possuir uma quantidade significativa de fotos, há o fenômeno da grande variabilidade de características nessa faixa etária. Enquanto nas crianças e idosos as características são sempre as mesmas – nas crianças, olhos maiores, peles mais lisas, etc. Nos idosos, rostos mais enrugados, orelhas maiores, etc – nos adolescentes, as características tendem a variar muito. Há adolescentes que aparentam ter 25 anos, outros que aparentam ter 10 anos, e outros que aparentam ter a idade que de fato tem. Isso pode ser observado analisando a matriz de confusão:

```
print(confusion_matrix(y_test, y_pred))
```

```
[[ 702   19  126    6]
 [  38   16  149    2]
 [  93   19 2390  110]
 [  11    4  225  247]]
```

Onde a diagonal principal representa a quantidade de verdadeiros positivos para cada predição e os demais componentes nas colunas representam os falsos positivos, para cada predição. Ou seja, para a categoria infantil, houve 702 acertos e $38+93+11 = 142$ erros, representando 83% de precisão. Ao observar a categoria dos adolescentes, percebe-se que houve uma confusão muito grande do algoritmo em classificá-los como de fato adolescentes, jogando alguns na categoria infantil e outros na categoria adulta, errando por muito pouquíssimas vezes (jogando somente 4 pessoas na categoria dos idosos), o que corrobora para a minha hipótese inicial. Em contrapartida, acertando muito nas crianças, adultos e idosos e errando por muito raramente na categoria idosos (jogando apenas 2% dos idosos nas categorias criança e adolescente, o que representa uma excelente predição).

Somando todos os acertos e dividindo pelo total, obtemos a acurácia geral do método:

```
print(accuracy_score(y_test, y_pred))
```

```
0.8070724079865288
```

Por fim, programei uma função para a predição de novas imagens nunca antes vistas pelo modelo. A saída do programa é uma foto 64x64 normal da pessoa, uma foto 64x64 reduzida pelo PCA e a categoria prevista para a pessoa na foto. Os testes podem ser encontrados a seguir:

1- Yuri Rocha – 22 anos



A imagem pertence à categoria: Adulto

2- João Victor – 20 anos



A imagem pertence à categoria: Adulto

3- Um bebe (foto pegada na internet)



imagem pertence à categoria: Criança

4- Um idoso (foto pegada na internet)



A imagem pertence à categoria: Idoso

Referências

scikit-learn - Documentação da biblioteca utilizada para os códigos: <https://scikit-learn.org/stable/>

ChatGPT - Recomendação de banco de dados de fotos e otimizações no código para diminuição do número de iterações e uso de memória (problema recorrente);

UTKFace – Conjunto de fotos usadas para o treino do modelo: <https://susanqq.github.io/UTKFace/>

Código - <https://github.com/yuriralb/previsaofaixaetaria>

Explicação sobre regressão logística multinomial - https://youtu.be/KbK_Nb9OS70